Graph Traversal Visualizer: Project Report

Project details

Graph Traversal Visualizer is a Design and Analysis of Algorithms (DAA) implementation project that runs BFS and DFS on a user-built graph and replays every step in the browser.

Item	Detail
Project title	Graph Traversal Visualizer: Interactive DAA Learning Platform
Subject	Design and Analysis of Algorithms (DAA)
Programme	B.Tech Artificial Intelligence and Machine Learning (CSE-AI&ML)
University	Marwadi University, Gujarat, India
Academic year	2026-2027
Division	5EN14

Team

Name	Division	Enrollment No.	Branch
Sri Sai Surya Vishwak Gadepally	5EN14	92400118494	CSE-AI&ML
Sushanth Pulicheri	5EN14	92400118445	CSE-AI&ML

Abstract and overview

The project teaches how Breadth-First Search (BFS) and Depth-First Search (DFS) explore a graph, one step at a time, using a real Python algorithm behind a web interface.

A student builds a graph on an SVG canvas, picks BFS or DFS and a start node, and presses Run traversal. The browser sends the graph as JSON to a Flask REST API. The Python backend runs the genuine algorithm, records a full snapshot of its state after every step, and returns those snapshots. The browser then replays them with play, pause, step forward and step backward, while the queue or stack, visited nodes, current node and a plain-English explanation update in sync.

The key design rule is that **no BFS or DFS code exists in JavaScript**. Python is the single source of truth, and the frontend only draws what the API returns.

Objectives and problem statement

Students often memorise BFS and DFS without seeing how the queue, stack and visited set change at each step, so the project makes that hidden state visible and interactive.

Objectives

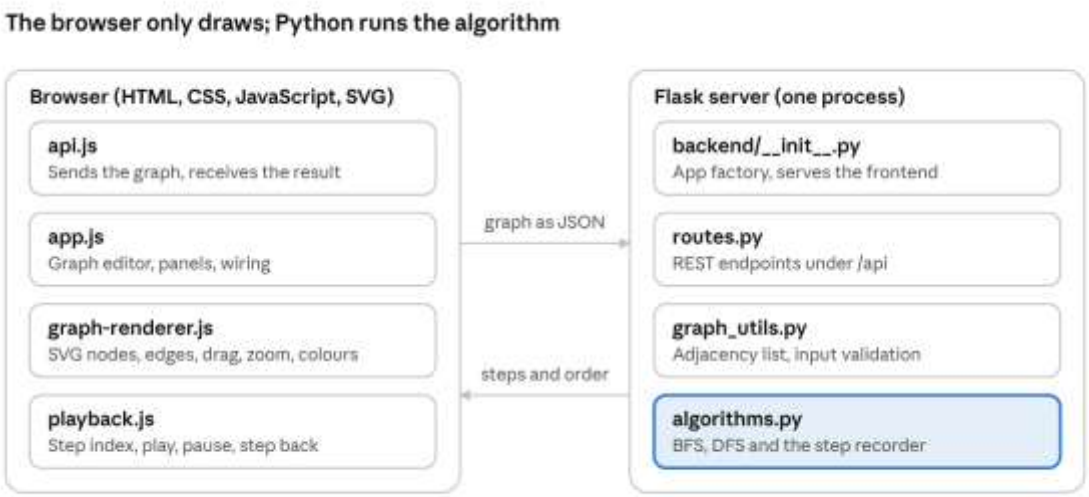
- Implement BFS and DFS correctly using an adjacency list, a queue, a stack and a visited set.
- Demonstrate O(V + E) time and O(V) auxiliary space, and explain why those bounds hold.
- Show how cycles and disconnected graphs are handled using visited tracking.
- Build a complete client-server application with a REST API, input validation and clear error messages.
- Verify correctness with automated tests, including a cross-check of iterative DFS against recursive DFS.
- Keep the code simple enough for either team member to explain in a viva.

Technology stack

Layer	Technology	Used for
Frontend	HTML5, CSS3	Page structure, responsive layout, animations, accessible focus states
Frontend	Vanilla JavaScript (ES6+)	Graph editor, playback controller, panels, no frameworks
Frontend	SVG	Nodes, edges, arrowheads, animated active edges, zoom and pan
Frontend	Fetch API	Calling the Flask REST API
Backend	Python 3	BFS, DFS, graph representation, step recording
Backend	Flask	REST API and serving the frontend from one server
Backend	Flask-CORS	Optional cross-origin support for the API
API	REST, JSON, GET and POST	Health, algorithm info, validation, traversal
Testing	pytest	59 automated tests
Fonts	Manrope, Inter	Headings and body text

System architecture

The browser sends the graph to Flask, Flask runs the algorithm and returns every step, and the browser replays them.



system architecture · browser and Flask server

The request is a POST to /api/graph/traverse. Because each returned step is a complete snapshot (current node, queue or stack, visited nodes, active edge), stepping backward needs no algorithm code in the browser.

Project structure

```
graph-traversal-visualizer/
  app.py                  entry point
  requirements.txt        Flask, Flask-CORS, pytest
  pytest.ini
  README.md
  backend/
    __init__.py           app factory, error handlers
    graph_utils.py        graph class, validation
    algorithms.py         BFS, DFS, step recorder
    routes.py             REST endpoints
  frontend/
    index.html
    css/style.css
    js/api.js, graph-renderer.js, playback.js, app.js
  tests/
    test_bfs.py, test_dfs.py, test_api.py
```

Data structures used

Four data structures do all the work, and each was chosen for a specific reason.

Need	Structure	Why this one
Store the graph	Adjacency list (dict: node to list of neighbours)	Scanning all neighbours of all nodes costs O(V + E), and memory is O(V + E), ideal for sparse graphs

Need	Structure	Why this one
BFS frontier	Queue (collections.deque)	First-in-first-out gives level-by-level order, and popleft() is O(1)
DFS frontier	Stack (Python list with append and pop)	Last-in-first-out keeps going deeper; implemented iteratively so each step can be recorded
Remember seen nodes	Visited set (set)	O(1) membership test prevents revisits and infinite loops on cycles

Neighbours are always ordered by the position of the node in the node list, so the same graph always produces the same traversal.

Algorithms

Both algorithms are written as taught in a DAA course; the only addition is a recorder that stores the full state after every action.

Breadth-First Search (queue, FIFO)

BFS visits the start node, then all its neighbours, then their neighbours, so it explores level by level. A node is marked discovered when it is enqueued, so it can never be queued twice.

```
BFS(G, s):
  create empty queue Q and visited set
  mark s visited;  enqueue(Q, s)
  while Q is not empty:
      u <- dequeue(Q)              // front
      visit(u)
      for each neighbour v of u in G[u]:
          if v not in visited:
              mark v visited       // mark on enqueue
              enqueue(Q, v)        // back
```

Depth-First Search (stack, LIFO)

DFS follows one path as deep as possible, then backtracks to the latest node that still has unexplored neighbours. The implementation is iterative: each stack entry has a pointer to its next neighbour. It produces exactly the same order as recursive DFS, and a test verifies that.

```
DFS(G, s):
  create empty stack S and visited set
  visit(s); mark s visited;  push(S, s)
  while S is not empty:
      u <- top(S)
      if u has an unexplored neighbour v:
          if v not in visited:
              visit(v); mark v visited;  push(S, v)   // go deeper
      else:
          pop(S)                                      // backtrack
```

Cycles

Before using a neighbour, the algorithm checks the visited set. If the neighbour was already seen, the edge is skipped and shown as a skip step. Every node is therefore processed exactly once and the algorithm always terminates.

Disconnected graphs

The user chooses one of two modes (the traverse_all setting in the API):

- **From the start node only** (default): visits just the reachable component and lists the rest as unreachable nodes.
- **Entire graph**: when the queue or stack empties, the algorithm restarts from the next unvisited node in creation order until every node is visited.

Complexity analysis

Both BFS and DFS run in O(V + E) time and use O(V) auxiliary space, where V is the number of vertices and E the number of edges.

Algorithm	Time complexity	Auxiliary space
BFS	O(V + E)	O(V)
DFS	O(V + E)	O(V)

Why O(V + E) time. Each vertex enters and leaves the queue or stack once, which is O(V). While handling a vertex we scan only its own neighbour list, and all lists together hold E entries for a directed graph (2E for undirected), which is O(E). An adjacency matrix would scan a full row of V cells per vertex and cost O(V squared).

Why O(V) extra memory. The visited set holds at most V nodes, and the queue or stack can also hold up to V nodes in the worst case. The adjacency list is the input, so it is not counted as auxiliary space. BFS memory peaks with the widest level; DFS memory peaks with the deepest path.

Theory versus measured runtime. Big-O describes how work grows with input size. The runtime_ms value returned by the API is only a stopwatch reading of a tiny input on one computer and is not the complexity. The edges-examined count equals E for directed graphs and 2E for undirected graphs, because each undirected edge is seen from both ends.

Backend and REST API

The Flask backend has four small modules, each with one job, and exposes four endpoints.

Module	Responsibility
app.py	Entry point that starts the Flask server
backend/**init**.py	App factory, JSON error handlers, serves the frontend
backend/graph_utils.py	Graph class, adjacency-list builder, request validation

Module	Responsibility
backend/algorithms.py	BFS, DFS and the step recorder (the DAA core)
backend/routes.py	REST endpoints under /api

Endpoints

Method	Endpoint	Purpose
GET	/api/health	API status and application name
GET	/api/algorithms	Algorithms, descriptions, data structures, complexity
POST	/api/graph/validate	Validate nodes, edges, start node and settings
POST	/api/graph/traverse	Run BFS or DFS; returns order, steps, counts, unreachable nodes, complexity

A traverse request carries the algorithm (bfs or dfs), the start node, a traverse_all flag and the graph (directed flag, nodes, edges). The response contains the traversal order, visited and unreachable nodes, the adjacency list, edges examined, backend runtime, and a list of step objects. Each step holds its type (enqueue, visit, skip, backtrack, restart or done), a plain-English action, the current node, the active edge, the queue or stack, the pending nodes and the visited list so far.

Error handling

Every error returns JSON with an error code and a readable message.

Situation	Code	HTTP
Algorithm not bfs or dfs	invalid_algorithm	400
Graph has no nodes	empty_graph	400
Start node missing or unknown	invalid_start_node	400
Duplicate node id	duplicate_node	400
Edge uses an unknown node	unknown_node_in_edge	400
Self-loop or duplicate edge	self_loop, duplicate_edge	400
Malformed JSON body	malformed_json	400
Wrong HTTP method	method_not_allowed	405
Unknown API route	not_found	404

Frontend design and features

The interface is a single responsive page with a warm, light theme built for readability of the graph rather than decoration.

Page sections

- **Navigation bar** with logo, links to Visualizer, Algorithms, Complexity and About, and a mobile menu.
- **Hero** with the heading "See how algorithms explore a graph.", two call-to-action buttons and an animated SVG graph with a highlighted path.
- **Visualizer** with the graph canvas, traversal setup, playback panel, adjacency list, result summary and BFS-versus-DFS comparison.
- **Algorithm cards** for BFS and DFS with explanation, a small worked example, pseudocode and use cases, plus an interactive comparison and a cycle-handling explanation.
- **Complexity** with the bounds table, the reasons, and a live box showing the current V, E and last measured runtime.
- **About** with the DAA concepts demonstrated and how browser, Flask and algorithms connect.

Graph editor

- Add a node by button or by double-clicking the canvas; nodes are placed in free space and cannot overlap when dragged.
- Add an edge by clicking two nodes; delete the selected node or edge with a button or the Delete key.
- Switch between directed (arrowheads, curved pairs for opposite edges) and undirected graphs.
- Load four sample graphs: simple connected, branching, cyclic and disconnected.
- Zoom with the wheel or buttons, pan by dragging the background, and reset the view.

Playback panel

The panel shows the selected algorithm, step number, progress bar, the current action in plain English, the current node, the queue or stack, visited nodes and the traversal sequence. Controls are reset, step backward, play or pause, and step forward, with a speed slider; the left and right arrow keys also step. Stepping backward works because every step from the backend is a complete snapshot.

Colour system

State	Colour
Page background	Warm ivory #FFF9F0
Headings, navigation, primary buttons	Deep forest green #234E3D
Current node	Golden amber #F2B84B
Visited node	Sage green #B9D7C0
Pending node (queue or stack)	Lavender #B9A7E8

State	Colour
Active edge	Coral #F07862
Body text	Charcoal #292D32

Headings use Manrope and body text uses Inter. The layout adapts to laptop, tablet and phone widths, and keyboard focus is clearly visible.

Testing and verification

All 59 automated pytest tests pass, and a real-browser run against the live Flask server passed every check.

Test file	What it checks
test_bfs.py	BFS order on a known tree; FIFO queue snapshots; single node; disconnected (component and entire-graph modes); cyclic graph terminates; directed graph; neighbour order; determinism; invalid start; invalid edge
test_dfs.py	DFS order on a known tree; goes deep before wide; LIFO stack snapshots; backtracking order; single node; disconnected; cyclic; directed; iterative DFS matches a recursive reference
test_api.py	Health and algorithms endpoints; response format for BFS and DFS; every validation error; malformed JSON; wrong HTTP methods; unknown route; frontend files served

Browser verification

A headless Chromium test drove the real page and confirmed that:

- the page loads and the API badge shows online;
- BFS and DFS orders shown in the page equal the orders returned by the API;
- step forward, step backward and reset work, and the panel's current node matches the highlighted graph node;
- editing the graph clears the old result, and an isolated new node is reported unreachable until entire-graph mode is switched on;
- edges can be added by two clicks, selected and deleted, and nodes can be dragged;
- directed mode shows arrowheads, and an empty graph shows the server's error message;
- the mobile layout has no horizontal overflow and the mobile menu opens.

Installation and running (Windows)

The app needs Python 3.9 or newer and runs as one Flask server serving both the website and the API.

Open PowerShell in the project folder and run:

```
python -m venv venv
venv\Scripts\activate
```

```
pip install -r requirements.txt
python app.py
```
Then open http://127.0.0.1:5000 in a browser. To run the tests, activate the virtual environment and run:

```
python -m pytest -v
```
If PowerShell blocks the activate script, run Set-ExecutionPolicy -Scope Process Bypass first, or use Command Prompt.

Skills used

The project combined algorithm design, full-stack development, UI design and testing skills, each applied in a specific part of the system.

Technical skills

Skill	Where it was applied
Design and analysis of algorithms	BFS and DFS, correctness reasoning (each node visited once), O(V + E) time and O(V) space analysis
Data structures	Adjacency list, queue (deque), stack (list), visited set, deterministic neighbour ordering
Graph theory	Directed and undirected graphs, cycles, connected components, unreachable nodes
Python programming	Algorithms, dataclasses, input validation, custom exception class, step recorder
Flask and REST API design	Blueprints, GET and POST endpoints, JSON schema, HTTP status codes, 400/404/405 error handlers, CORS
Input validation and error handling	Duplicate nodes, unknown edge endpoints, self-loops, malformed JSON, invalid algorithm or start node
JavaScript (ES6+)	Classes, async/await, Fetch API, event handling, state management, DOM building without frameworks
SVG graphics	Drawing nodes and edges, arrowheads, curved edges, drag, zoom and pan with viewBox, animated active edges
HTML5 and CSS3	Semantic structure, CSS variables, grid and flexbox, responsive breakpoints, transitions and keyframe animations
Client-server integration	Browser sends the graph as JSON, backend returns steps, frontend replays them without re-running the algorithm

Skill	Where it was applied
Software testing	pytest suites, known-answer tests, FIFO and LIFO checks, cross-checking against a recursive reference, API tests
End-to-end testing	Headless Chromium run against the live server, including a phone-width layout check
Project setup and tooling	Virtual environment, requirements file, pytest configuration, Windows run instructions

Design and usability skills

- **UI and UX design:** a custom warm colour palette with consistent state colours, clear visual hierarchy and a playback panel that explains every step.
- **Responsive and accessible design:** mobile menu, visible keyboard focus, skip link, reduced-motion support, screen-reader labels on controls and the canvas.
- **Data visualisation:** colour-coded node states, order badges, traversal-tree edges and an animated active edge.
- **Educational design:** pseudocode, worked examples, cycle explanation and a clear split between theoretical complexity and measured runtime.

Professional skills

- **Teamwork and division of work** between the two team members across backend, frontend, testing and documentation.
- **Technical writing:** README, API documentation, this report and viva preparation notes.
- **Problem solving and debugging:** for example, fixing a mobile overflow found during browser testing and a route clash that hid HTTP 405 errors.
- **Presentation and viva readiness:** every part is simple enough to explain and defend.

Limitations and future scope

The project handles unweighted graphs of up to 30 nodes in the editor, and there are clear next steps.

Current limitations

- Edges have no weights, so shortest paths in weighted graphs (Dijkstra) are not covered.
- Self-loops and parallel edges are rejected.
- The editor limits the graph to 30 nodes for readability, while the API accepts up to 200.
- Node positions come from the user or the sample graph; there is no automatic force-directed layout.
- Individual nodes cannot yet be selected by keyboard, although the editing buttons, step keys and Delete key work.

Future scope

- Weighted graphs, Dijkstra and shortest-path highlighting.

- Cycle detection and topological sort built on DFS.
- A recursion-tree view of DFS.
- Saving and loading graphs, and automatic layout.
- Keyboard selection of nodes and more languages for step explanations.

Conclusion

The Graph Traversal Visualizer meets its goal: a complete, tested web application in which real Python BFS and DFS drive an interactive, step-by-step visual explanation.

It demonstrates the central DAA ideas of adjacency-list representation, queue and stack behaviour, visited-set cycle handling and O(V + E) analysis, while also showing how a REST API connects a browser to server-side logic. With 59 passing tests and a verified end-to-end run, the project is ready to demonstrate and defend in a viva.

Appendix: viva questions and answers

What is graph traversal? Visiting every node reachable from a start node in a systematic way, exactly once. BFS and DFS are the two basic strategies.

What is the difference between BFS and DFS? BFS explores level by level using a queue and finds shortest paths in unweighted graphs. DFS goes as deep as possible and then backtracks, using a stack or recursion, and suits cycle detection and backtracking problems.

Why does BFS use a queue? Nodes must be processed in the order they were discovered, so closer nodes go before farther ones. A queue is first-in-first-out.

Why does DFS use a stack? We must continue from the most recently discovered node to keep going deeper. A stack is last-in-first-out, and recursion uses the call stack for the same effect.

What are V and E? V is the number of vertices (nodes) and E is the number of edges.

Why is the time complexity O(V + E)? Each vertex enters and leaves the queue or stack once, and each adjacency list is scanned once, so the total work is V plus E (or V plus 2E when undirected).

How are cycles handled? With a visited set. A neighbour that is already visited is skipped, so every node is processed once and cycles cannot loop forever.

Why use an adjacency list? It stores only existing edges, so memory and traversal are O(V + E). A matrix needs O(V squared) for both, which wastes space on sparse graphs.

Why is a node marked when enqueued in BFS? Otherwise, the same node could be queued several times before being processed, wasting time and breaking the O(V + E) bound.

What is a REST API? A way for programs to talk over HTTP using URLs and standard methods such as GET and POST, usually with JSON, where each request carries everything, the server needs.

How does the browser communicate with Flask? JavaScript's fetch() sends an HTTP POST with the graph as JSON to /api/graph/traverse; Flask runs the matching function and returns JSON, which JavaScript turns into the animation.

How did you test the algorithms? With pytest: known graphs with hand-checked orders, FIFO and LIFO checks on recorded snapshots, edge cases (single node, disconnected, cyclic, directed), a comparison with a recursive DFS, and API tests for every success and error case.